

Transformer Zoo: A Comprehensive Analysis of Transform Coding in Image Compression

1st Luca Araujo

Computer Science Department
Carleton College
Northfield, United States
araujol@carleton.edu

2nd Joshua Lee

Computer Science Department
Carleton College
Northfield, United States
leej12@carleton.edu

3rd Indy Lyness

Computer Science Department
Carleton College
Northfield, United States
lynessi@carleton.edu

4th Jiale Wan

Computer Science Department
Carleton College
Northfield, United States
wanj2@carleton.edu

Abstract—The JPEG Compression Standard is a staple in modern image compression that provides a robust pipeline for transforming, quantizing, and encoding images. In this paper we define the basic structure of the JPEG pipeline, then explain how transforms other than the standard DCT (Discrete Cosine Transform) can be substituted in, covering the Discrete Fourier Transform (DFT) and the Discrete Wavelet Transform (DWT), represented by the Haar and S+P Transform. We provide a rich mathematical background for each transform and then compare their performance across several datasets.

I. INTRODUCTION

As file sizes continue to increase, efficient compression algorithms become more important in order to keep up with the demand for transmitting large amounts of information. Images, a crucial datatype found all across the web, is a prime example of data that requires compression, especially as increasing display resolution pushes the need for higher (and more space-demanding) resolution images [9].

Data compression is the process of representing data in a form that occupies less space than the raw data. Compressed data is found everywhere today; almost all files that users interact with are stored in a compressed format. There are two types of data compression: lossless compression and lossy compression. Lossless compression involves representing the original data such that no information is changed or lost, with common examples being *.zip* and *.tar* files. Lossy compression, as the name suggests, permanently alters the information in order to achieve higher compression. In the case of images, lossy compression aims to discard the information that is least perceptible to humans so that compression can be more efficient while maintaining image quality. This way, the information that is lost via lossy compression is information that the human visual perception system does not need in order to recognize the image.

One of the most widely used image compression standards is the JPEG Compression Standard, which has been in use for over 30 years [24]. This paper deals with the JPEG Compression Standard, exploring its use of different compression techniques and then tweaking its use of transform coding, a crucial part of the compression process implemented in JPEG. Originally utilizing the Discrete Cosine Transform (DCT), this project dives into other transforms by using different

transforms in the JPEG pipeline to investigate their effects on various metrics used to determine the efficiency of an image compression algorithm.

II. LITERATURE REVIEW

Claude Shannon’s introduction of entropy in the context of information theory sets the stage for much of the theoretical discussion about data compression [20]. Entropy is an important metric used to calculate the upper bound for how much a piece of data can be compressed losslessly.

a) *Empirical entropy of coefficient streams.*: For a discrete stream of symbols $\{s_1, s_2, \dots, s_n\}$ with frequency of occurrence $\{p_1, p_2, \dots, p_n\}$, we compute the Shannon entropy as

$$H = - \sum_{k=1}^n p_k \log_2 p_k, \quad (1)$$

which serves as an information-theoretic lower bound on the average number of bits needed for lossless coding of that stream. Not limited to just image compression, entropy describes how “random” a string of data is, which directly correlates to the maximum compression ratio it is possible to achieve, where compression ratio is calculated by

$$\text{compression ratio} = \frac{\text{original filesize}}{\text{compressed filesize}}. \quad (2)$$

The JPEG Compression Standard and its specifications are defined in various publications, with Wallace’s 1992 paper describing its pipeline [24]. A more in-depth description of specific implementation techniques used in JPEG is published by Purdue University [1]. The mathematical theory and additional signal processing background is provided in Broughton to provide a basis for both the DFT and DCT [4]. The analysis and theory of Wavelet Transforms, both with Haar basis and more complex examples are discussed by Stollnitz et al. [22] [23]. The S+P Transform extends on the ideas of the Haar wavelet, with the original paper released in 1993 [17].

III. METHODS

A. Pipeline

Our pipeline follows the standard structure of a JPEG-style system to test transform coding, shown in Fig. 1 (Image Compression Pipeline). Each stage prepares the image for the

next, allowing us to isolate the effect of different transforms while keeping the rest of the system fixed.

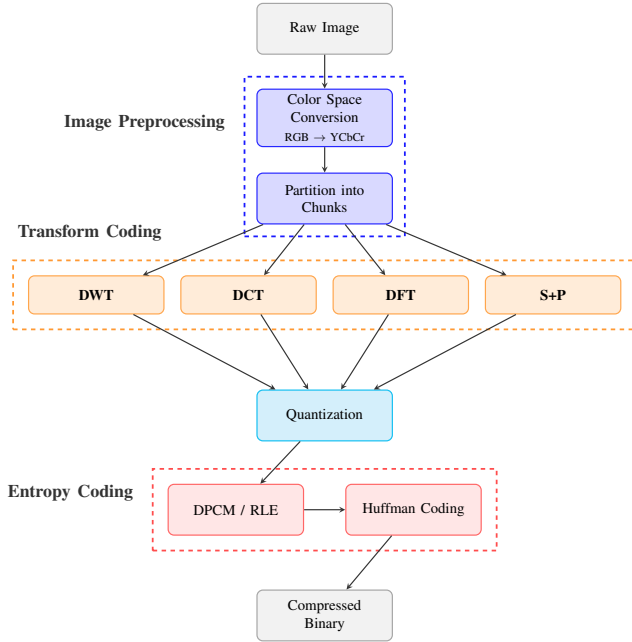


Fig. 1: Image compression pipeline.

We begin with a raw RGB image. Because RGB channels are highly correlated, we first convert the image into YCbCr, where luminance (Y) is separated from chrominance (Cb, Cr). This step concentrates most visually important information into a single channel, making subsequent compression more effective. Next, the image is partitioned into blocks as per the specific transform (e.g., 8×8 for DCT/DFT, full-image tiles for Haar and S+P).

Then, the transform stage is the core of our investigation. Here we substitute one of four transforms (DCT, DFT, Haar, or S+P), while keeping all later steps unchanged, with the exception of routine subband-aware quantization for S+P (described in Section III-F). Each transform maps spatial pixel values into a domain where redundancy is more explicit: DCT and DFT concentrate energy into low-frequency coefficients; Haar and S+P build a multiresolution decomposition. All transforms produce a set of coefficients organized into subbands or frequency components.

These coefficients then undergo quantization, the primary source of loss in the pipeline. Quantization is a nonlinear operation that partitions the range of possible input values into disjoint intervals and assigns each interval a single representative value (the reconstruction level). All inputs falling within the same interval are encoded identically, which reduces precision and introduces quantization error. Because transforms ideally decorrelate the signal and push most meaningful information into a small number of coefficients, quantization can discard many small values with limited perceptual impact. This makes quantization the crucial stage where transform quality directly affects rate-distortion behavior.

The quantized coefficients are finally passed into an entropy coding module consisting of prediction (DPCM or linear predictive coding), run-length encoding (RLE) of zeros, and Huffman coding. These operations are lossless and convert long runs of similar or repeated values, introduced by quantization, into compact binary representations.

The output of entropy coding forms the compressed bitstream. Decoding inverts each stage in reverse order. Keeping (virtually) all components except the transform fixed, this pipeline allows us to isolate how each transform influences entropy, compressibility, and reconstruction quality across datasets.

B. Background

Before diving into the individual transform methods, we first explain what transform coding is and why it helps with compression. The idea is to take the original representation of our image, a 2D array of pixel values, and transform it into a new space with lower entropy. The hope is that the result will be easily compressed by the entropy encoding techniques described in Section III-G.

We do this by considering some original signal, which we will denote as

$$x = \{x_1, x_2, \dots, x_N\}. \quad (3)$$

The signal can be thought of as a vector in the vector space $\mathbb{R}^N$. Loosely speaking, a vector space V is any collection of elements where

- 1) For all $a, b \in \mathbb{R}$ and $u, v \in V$, $au + bv \in V$.
- 2) There is a unique element $0 \in V$ such that for all $v \in V$, $0v = 0$ and $v + 0 = v$.

In other words, it is a space that respects and is closed under the normal operation of addition and multiplication on the real numbers $\mathbb{R}$. Vector spaces have many rich theoretical properties, for which the interested reader can refer to the textbook by Broughton [4]. We will only cover the most essential properties here. A set of vectors $v_1, v_2, \dots, v_k$ is called *linearly independent* if the only solution to the equation

$$c_1v_1 + c_2v_2 + \dots + c_kv_k = 0 \quad (4)$$

is that $c_i = 0$ for $i = 1, 2, \dots, k$. A set of linearly independent vectors $v_1, v_2, \dots, v_N$ is called a *basis* if for any vector $u \in V$, there exists numbers $c_1, c_2, \dots, c_N$ such that

$$u = \sum_{i=1}^N c_i v_i. \quad (5)$$

In other words, every member of the vector space can be represented as some weighted sum of vectors in a basis. These weights $c_1, c_2, \dots, c_N$ are called the coefficients. Fig. 2 illustrates this with one of the simplest examples $\mathbb{R}^2$, where the vectors $\langle 0, 1 \rangle$ and $\langle 1, 0 \rangle$ form a basis. In this example, we get the equation $v = 3i + 2j$ where 3 and 2 are our coefficients. Notably, for any given vector space there are many possible bases. The transform algorithms described in this section are

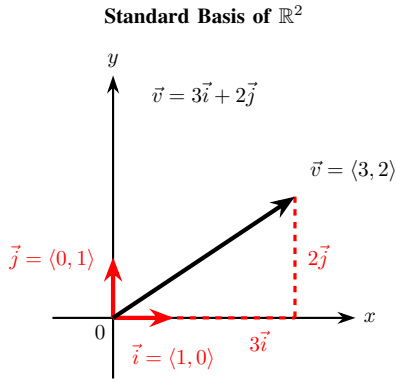


Fig. 2: The vectors $\vec{i}$ and $\vec{j}$ form a 2D basis of $\mathbb{R}^2$

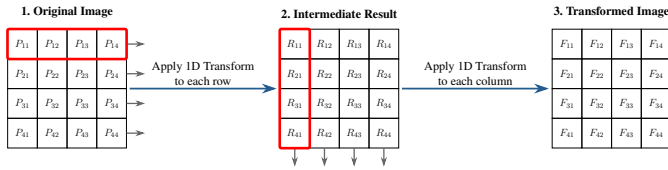


Fig. 3: Applying a transform to an image

all effectively a *change of basis*, where the new, transformed signal is denoted

$$X = \{X_1, X_2, \dots, X_N\}. \quad (6)$$

These new values can be thought of as the coordinates for the signal in the new transformed space. By cleverly choosing a basis, the hope is that more information of the original data is encoded in fewer of the new coefficients.

Applying this to images specifically, if we represent our image as a 2D array of pixels, the resulting transformed image can be acquired by applying the transform to each row and column, as demonstrated in Fig. 3.

C. Discrete Wavelet Transform

Luca

The Discrete Wavelet Transform (DWT) is a powerful transform coding technique that decomposes signals into a hierarchy of resolution levels, making it particularly well-suited for image compression. In this work we focus on the Haar wavelet, which is the simplest in the family of wavelets used for this transform. For now on we consider an image as a 1D signal, we later discuss how to convert it to 2D.

1) *One Dimensional Haar Wavelet Transform in Practice:* Consider a 1D signal with 2^k values:

$$\{x_1, x_2, \dots, x_{2^k}\}$$

A naive approach to compression might be to store only the average of adjacent values, resulting in the compressed signal

$$A_{k-1} = \{(x_1 + x_2)/2, (x_3 + x_4)/2, \dots, (x_{2^{k-1}} + x_{2^k})/2\}$$

Note that this approach clearly loses information. To enable perfect reconstruction of the original signal, we must also preserve the difference between adjacent values:

$$D_{k-1} = \{x_1 - (x_1 + x_2)/2, x_3 - (x_3 + x_4)/2, \dots\}$$

Note that the difference of x_i to the mean is completely symmetric to the difference of x_{i+1} to the mean, so we only need to store one detail coefficient. This simple idea is the core of the Haar wavelet transform.

We can apply this operation once and we get a set of $2^k/2$ average coefficients and $2^k/2$ detail coefficients. We can keep applying the operation recursively to the average coefficients until we reach only one pixel of average coefficient. Table I illustrates this process for an 8-pixel signal.

Figure 4 visualizes this decomposition process, showing the signal (blue) and wavelets (red) at each resolution level. The blue curves represent the averaged signals at decreasing resolutions, while the red curves capture the detail coefficients (differences) that must be preserved for perfect reconstruction.

TABLE I: Haar decomposition example for an 8-pixel signal

Resolution	Averages	Details
8	[5, 8, 9, 7, 4, 2, 1, 3]	—
4	[6.5, 8, 3, 2]	[-1.5, 1, 1, -1]
2	[7.25, 2.5]	[-0.75, 0.5]
1	[4.875]	[2.375]

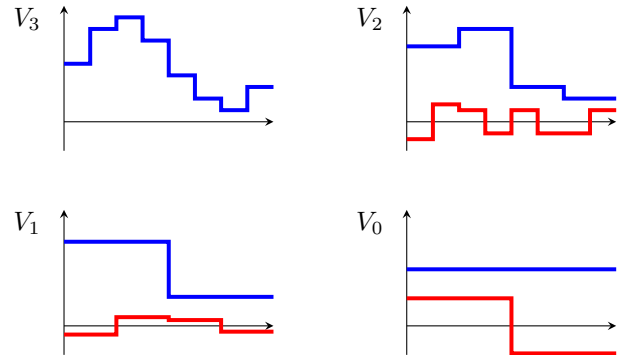


Fig. 4: Haar wavelet decomposition showing signal (blue) and wavelets (red) at each resolution level.

Note that for a given level we can perfectly reconstruct the previous level's average coefficients from the combination of the average coefficient and detail coefficients, so all we need to store in a compression is the set of all detail coefficients and the very last average.

2) *Theoretical Foundation for Haar Wavelet Transform:* The Haar wavelet transform is founded on a set of nested vector spaces V^j representing piecewise-constant functions at different resolutions, where $V^0 \subset V^1 \subset V^2 \subset \dots$. This nested structure enables a change of basis from the natural basis of scaling functions in V^{j+1} to a basis consisting of the scaling functions from the lower-resolution space V^j together with the wavelet basis functions from the orthogonal

complement space W^j . The scaling functions provide the coarse approximation, while the wavelets capture the detail information needed for perfect reconstruction. Figure 5 visualizes the scaling functions and Haar wavelets for different resolution levels.

A detailed mathematical exposition of the vector space structure, scaling functions, inner products, orthogonality, and the Haar wavelet basis functions is provided in Appendix A.

3) *Two-Dimensional Haar Wavelet Transform*: The 2D Haar wavelet transform generalizes the 1D "averaging and differencing" filter bank to 2D by alternating between operations on rows and columns at each level of the hierarchy.

a) *The Decomposition Algorithm*: The algorithm proceeds recursively as follows:

- 1) **Row Step**: First, perform one step of horizontal pairwise averaging and differencing on the pixel values in each row of the image. This splits the image horizontally into averages (left) and details (right).
- 2) **Column Step**: Next, apply vertical pairwise averaging and differencing to each column of the result from the Row Step.
 - This creates four distinct quadrants of coefficients:
 - **Top-Left**: Averages of averages (coarse approximation).
 - **Top-Right**: Averages of horizontal details.
 - **Bottom-Left**: Vertical details of horizontal averages.
 - **Bottom-Right**: Vertical details of horizontal details (diagonal details).
- 3) **Recursion**: To complete the transformation, repeat this process recursively *only* on the quadrant containing the averages in both directions (the top-left quadrant).

Figure 6 visualizes this 2D Haar transform decomposition process, showing how an image is decomposed into four quadrants at each level.

4) *Compression Properties*: The Haar wavelet transform is particularly effective for image compression because it naturally concentrates energy in the low-resolution approximation coefficients. The detail coefficients (wavelets) typically have smaller magnitudes and can be more aggressively quantized or discarded in lossy compression scenarios. This energy compaction property, combined with the transform's computational efficiency makes the Haar wavelet an attractive choice for practical image compression systems.

D. Discrete Fourier and Cosine Transforms

Two of the most prevalent transforms in signal processing are the Discrete Fourier Transform (DFT) and its derivative, the Discrete Cosine Transform (DCT). Broadly speaking, these transforms represent the original input as a sum of different wave frequencies. We will first discuss the DFT and its shortcomings, then show how this naturally gives rise to the DCT.

1) *The Discrete Fourier Transform in 1 Dimension*: For the DFT, we consider our input as a 1-dimensional signal $x = \{x_0, \dots, x_{N-1}\}$ as described in Section III-B¹. For Fourier Analysis, it is common to call x a **time signal**, which is any ordered list of values (in our case, the pixels along the rows and columns of the image). We assume that our signal is periodic modulo N , extending to infinity using the relation $x_k = x_{Nm+k}$ for each $m \in \mathbb{Z}$ as shown in Fig. 7. This way, we can define our basis to be the set of complex waveforms

$$E = \{E_0, E_1, \dots, E_{N-1}\}, \quad (7)$$

where

$$E_k = \{e^{2\pi i k(0)/N}, e^{2\pi i k(1)/N}, \dots, e^{2\pi i k(N-1)/N}\}. \quad (8)$$

As a reminder, complex numbers are of the form $a + bi$, where $a, b \in \mathbb{R}$ and $i = \sqrt{-1}$. Also, we have Euler's identity

$$e^{i\theta} = \cos \theta + i \sin \theta. \quad (9)$$

This means that the basis is effectively made of discrete rotations around the unit circle in the complex plane at different frequencies. An example for the basis E_1 for $N = 8$ is shown in Fig. 8. Note that the basis starts at 1 and travels around the unit circle clockwise.

All of the basis vectors for $N = 8$ are represented in Fig. 9, with both the real part (blue) and imaginary part (red). We have chosen $N = 8$ for these figures as this will be our choice of block size for the 2D case later in this section. Note that the first basis vector E_0 is constant, while each subsequent vector has increasing frequency.

We can transform the signal into a representation in our new basis using the following formula:

$$X_k = \frac{1}{\sqrt{N}} \sum_{m=0}^{N-1} x_m e^{-2\pi i k m / N}. \quad (10)$$

This forms a new signal $X = \{X_0, X_1, \dots, X_{N-1}\}$ (note that the $1/\sqrt{N}$ term is just a normalization). We say that X lives in the **frequency domain**. With the change of basis, each X_k is now the corresponding coefficient for the basis vector E_k . The first coefficient X_0 is called the **DC coefficient**, and corresponds to the average value of the signal since E_0 is constant. All other coefficients are called **AC coefficients**². This transform is lossless and invertible, with each value in the original time signal retrieved by calculating

$$X_k = \frac{1}{\sqrt{N}} \sum_{m=0}^{N-1} X_m e^{2\pi i k m / N}. \quad (11)$$

We call equations 10 and 11 the **forward** and **inverse** transforms, respectively. These formulas are derived from taking the normal **inner product** in the vector space $\mathbb{C}^N$. Those interested in the mathematical background should refer to Broughton [4].

¹We shift the indices to start at 0 throughout this section for convenience.

²The terms DC and AC come from an electrical context, meaning *direct current* and *alternating current*

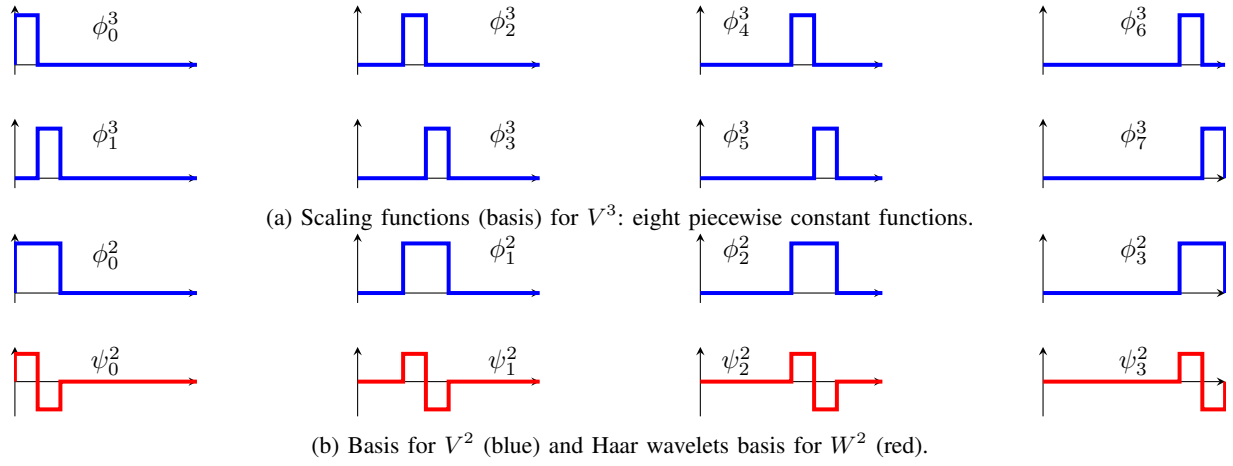


Fig. 5: Haar basis: scaling functions for V^3 and scaling functions and wavelets for V^2 and W^2 .



Fig. 6: Visualization of a 2D Haar transform showing the decomposition of an image

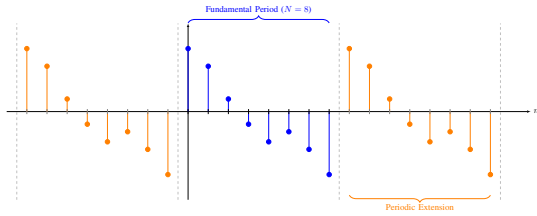


Fig. 7: The signal extension of $x \bmod N$

2) *Applying the Discrete Fourier Transform to a 2D Image:* We now discuss how to apply the 1D DFT to an image. As discussed in Section III-B, we represent the image as a 2D array of pixels in the YCbCr colorspace described in Section I. We then break the image up into 8×8 blocks of pixels, and apply the 1D DFT along each row and each column of the block. A visualization for the transform is provided in Fig. 10.

In Section III-B we mentioned that the goal of the transform was to reduce entropy as defined in Section I. If the image

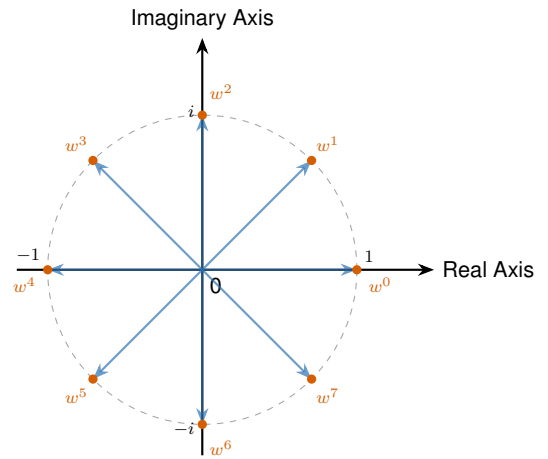


Fig. 8: The basis E_1 for $N = 8$ in the complex plane. Note that these are all the solutions to $\omega^8 = 1$, the so-called 8th roots of unity.

was random noise, we would generally not expect this to be the case. However, since pixels are correlated with those around them in an image, we don't expect sharp changes in pixel values. This means in our new basis, the coefficients for the lower frequency basis vectors $E_0, E_1, E_2, \dots$ have larger values than the coefficients for the higher frequency basis vectors $\dots, E_{N-2}, E_{N-1}$. This results in a lower entropy, which combined with quantization discussed in Section III-F allows for solid compression ratios.

3) *Problems with the DFT:* At this point, we have built up the DFT as a change of basis on an image to represent it as frequency components where we expect the energy of the signal to be concentrated in lower frequency coefficients. However, there are several problems that arise that greatly limit the ability of the DFT to achieve great compression.

- 1) The DFT is slow. A raw application of Equation 10 requires $\mathcal{O}(N^2)$ complex multiplications, which means

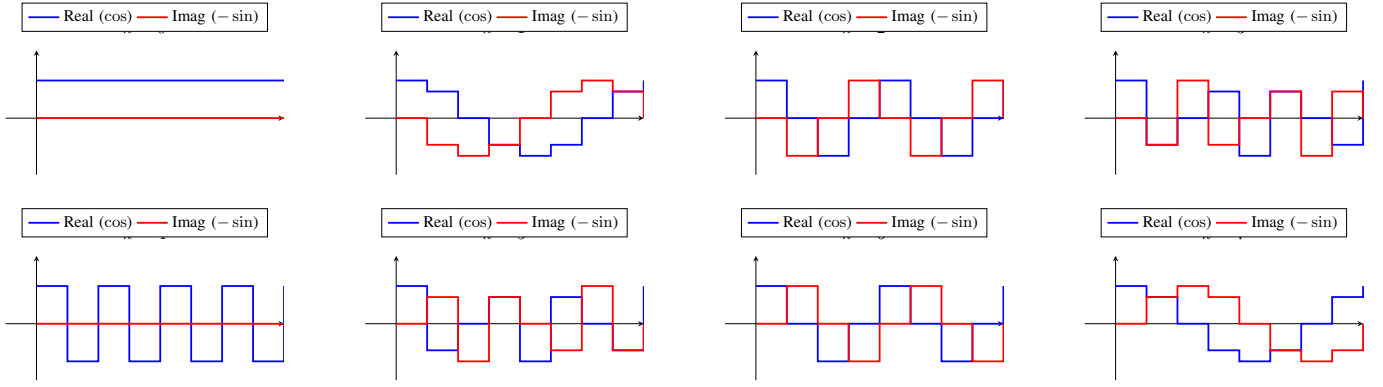


Fig. 9: Every DFT basis vector for $N = 8$. The first row is E_0, E_1, E_2, E_3 , and the second row is E_4, E_5, E_6, E_7 .



Fig. 10: An original Image from the Kodak dataset (left) and a visualization after applying the DFT (right). The image is 512×512 pixels, and at a closer look one can see block-like shapes where the transform has been applied to 8×8 chunks.

for a 2D Image the runtime is $\mathcal{O}(NM)^2$, for an $N \times M$ image (or just $\mathcal{O}(N^4)$ if we assume that the image is square).

- 2) The output to the DFT is complex-valued. Since we are using complex waveforms as a basis, we are thinking of our pixels as living in the complex plane $\mathbb{C}$, not the real numbers $\mathbb{R}$. Complex values take twice as much space to store as real numbers.
- 3) The DFT does not fully eliminate large values in high frequency coefficients. Since the DFT assumes the signal x is periodic modulo N , if $|x_0 - x_{N-1}|$ is large, then there will be a high frequency jump in the signal resulting in larger terms (Refer back to Fig. 7 for an example).

The first issue of runtime can be addressed by using a clever algorithm called the Fast Fourier Transform (FFT), which exploits the symmetries of complex waveforms to reduce the runtime to $\mathcal{O}(N^2 \log N)$ for an $N \times N$ pixel image. More details are provided in Appendix D. At first glance, the following issues are not so easy to fix. However, by changing the assumptions about our signal x slightly, we are able to solve both problems simultaneously. This is exactly what the Discrete Cosine Transform (DCT) does.

4) *The Discrete Cosine Transform:* The idea behind the DCT is very simple. Starting with our signal $x = \{x_0, \dots, x_{N-1}\}$, we first **mirror** our signal to one of length

$2N$, giving a new signal

$$\hat{x} = \{x_0, \dots, x_{N-1}, x_{N-1}, \dots, x_0\}, \quad (12)$$

which we then assume is periodic modulo $2N$ instead as shown in Fig. 11 in the $N = 8$ case. We then perform the DFT on this extended signal to get a transformed signal $\hat{X} = \{X_0, \dots, X_{2N-1}\}$. With a little bit of unpacking, one can start with Equation 10 and show that the coefficients for the resulting transform are given by

$$X_k = \sum_{m=0}^{2N-1} x_m e^{-2\pi i k m / 2N} \quad (13)$$

$$= 2e^{2\pi i k / 2N} \sum_{m=0}^{N-1} x_m \cos\left(\frac{\pi k(m + \frac{1}{2})}{2N}\right), \quad (14)$$

where we have taken advantage of the symmetry in our mirrored signal $\hat{x}$. These terms are almost real, and after applying a phase shift and normalizing we get the final DCT coefficients given by

$$X_k = \sqrt{\frac{c_k}{N}} \sum_{m=0}^{N-1} x_m \cos\left(\frac{\pi k(m + \frac{1}{2})}{N}\right), \quad (15)$$

taking $c_k = 1$ if $k = 0$ and $c_k = 2$ otherwise. Note that this gives us only half of the $2N$ coefficients that we expect. This is sufficient because due to the symmetry of the signal, the remaining coefficients are recovered by the identity

$$X_{2N-r} = -X_r \quad \text{for } r = 0, 1, \dots, N. \quad (16)$$

Notice that this forces $X_N = 0$. The math behind these relations can be found in Broughton [4]. We now notice the 2 critical facts about the result.

- 1) The DCT is real-valued. The mirrored nature of the extended signal $\hat{x}$ has washed out all imaginary components.
- 2) The new signal also has no potential for sharp gaps when moving to a new period, meaning smaller values in the higher frequency coefficients.

These are exactly the two problems that we wanted to improve on for the DFT, and we see that the DCT achieves both by

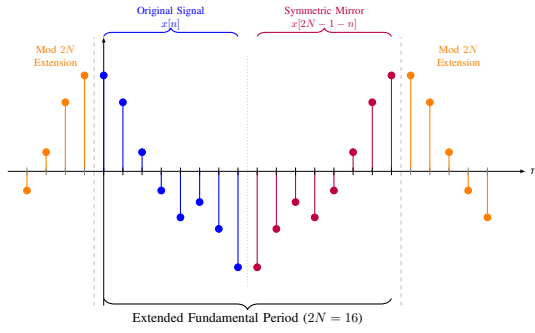


Fig. 11: The new signal $\hat{x}$ extended modulo $2N$ for $N = 8$. Note that there are no sharp gaps at the extension point.

mirroring the signal and using the relation in Equation 16 to not increase space. To apply the DCT to an image, one can simply follow the same steps for the DFT, except to mirror each row and each column of pixels in each chunk before applying the transform, and saving the first half of the result. See Section IV for just how much more effective the DCT is over the DFT.

E. The S+P Transform

1) *Motivation and High-Level Overview:* To address the need for invertibility without inflating the dynamic range in the process of compression, the S+P transform (Sequential transform + Prediction), originally proposed by Said and Pearlman [17], employs a simple combination of a Haar-like multiresolution transform (the “S” stage) and an embedded predictive coding stage (the “P” stage).

In the broader context of our project, we treat S+P as a hybrid transform that lives between classical wavelet/subband decompositions(transform) and purely predictive schemes. Earlier sections introduce transform coding and the Haar wavelet; here, we focus on what S+P adds on top of that baseline, and how those additions behave inside our JPEG-style pipeline.

Operationally, S+P still proceeds along one-dimensional signals (rows or columns of an image). The S stage performs an integer lifting-style average/difference, producing lowpass (approximate local averages) and highpass (local differences). The P stage then applies an *integer* prediction filter directly to the highpass branch, aiming to decorrelate it further before quantization and entropy coding. Recursively applying S+P to the lowpass component yields the same LL/HL/LH/HH multiresolution layout as Haar, but with much stronger whitening in the highpass bands and stricter control over rounding and borders.

2) *Sequential Transform (S) Recap:* We briefly recall the 1D S stage, which is the integer analogue of the Haar lifting step. Let $c[n]$ be a sequence of integer samples of length N . For each pair $(c[2n], c[2n+1])$ we define

$$d_1[n] = c[2n+1] - c[2n], \quad (17)$$

$$s[n] = c[2n] + \left\lfloor \frac{d_1[n]}{2} \right\rfloor, \quad (18)$$

for $n = 0, \dots, \lfloor N/2 \rfloor - 1$. Here $s[n]$ acts as a local average, while $d_1[n]$ stores the corresponding difference as detail.

This mapping is exactly reversible with integer arithmetic:

$$c[2n] = s[n] - \left\lfloor \frac{d_1[n]}{2} \right\rfloor, \quad (19)$$

$$c[2n+1] = c[2n] + d_1[n]. \quad (20)$$

No fractional bits are introduced: only additions, subtractions, and integer floor-divisions by powers of two are used. Applying this S step to all rows and then all columns, and recursing only on the LL quadrant, gives the same multilevel LL/HL/LH/HH pyramid described for the Haar transform, but now entirely in integers. As shown in Figure 12, each level of S+P transform decompose the image into the four subbands: LL carries the coarse approximation as the mean image of the original, HL captures vertical edges, LH captures horizontal edges, and HH captures fine diagonal texture (the highpass details are scaled up to be more visible). After the second level of decomposition, the LL band can be transformed again to give a multiresolution pyramid, as shown in Figure 13. The recursion would continue until one dimension cannot be halved.

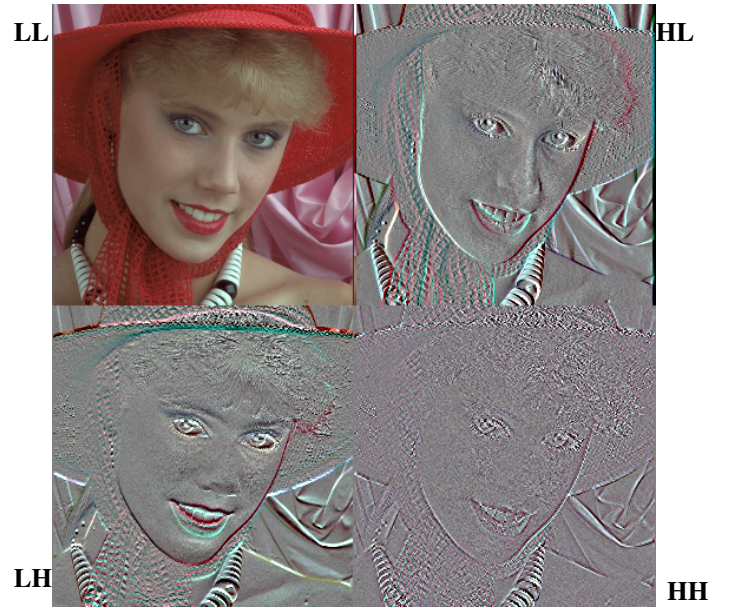


Fig. 12: One level of the S+P transform.

The correlation coefficient measures the normalized linear dependence between two adjacent data points, ranging from -1 to 1, with higher positive values indicating stronger similarity. Under mild correlation assumptions—in particular, when adjacent samples have correlation coefficient larger than $\frac{1}{3}$ —S alone already reduces the average variance of the low- and highpass components relative to the original signal [17]. The P stage is specifically designed to exploit the residual correlation that remains, mainly in the highpass bands.

3) *The S+P Transform: Prediction Embedded in the Transform:* The S+P transform augments the S stage with a predictive step that operates directly on the highpass coefficients

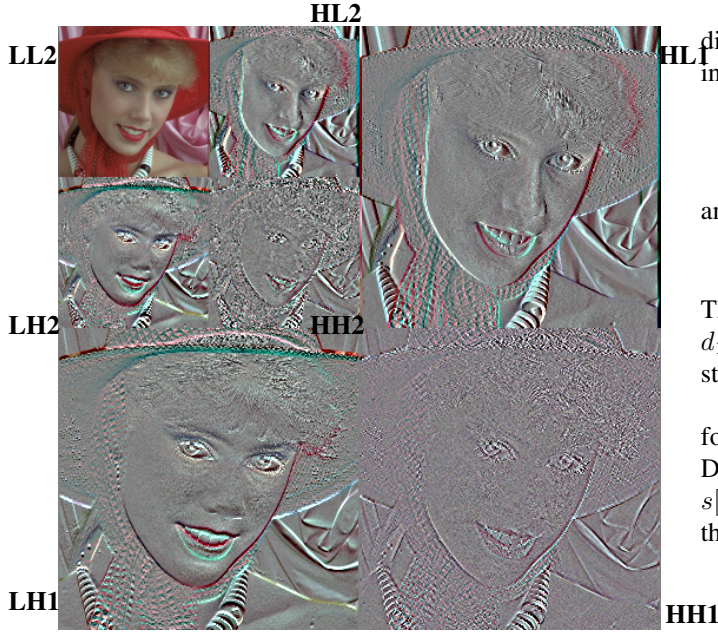


Fig. 13: Two levels of the S+P transform.

produced at each one-dimensional pass. Conceptually, we distinguish three sequences:

- $s[n]$: the lowpass component after the S stage,
- $d_1[n]$: the intermediate highpass component (exact differences),
- $d[n]$: the final highpass residual after prediction.

4) *Predictor structure*: We introduce a simple notation for local lowpass differences:

$$z_\ell[n] = s[n-1] - s[n], \quad (21)$$

where ℓ captures local gradients in the lowpass band. Following [17], we estimate $d_1[n]$ from neighboring lowpass samples and the next highpass value $d_1[n+1]$. A generic linear estimator has the form

$$\hat{d}_1[n] = \beta_{-1} z_\ell[n] + \beta_0 z_\ell[n+1] + \beta_{+1} z_\ell[n+2] - \phi_1 d_1[n+1]. \quad (22)$$

where $\beta_{-1}, \beta_0, \beta_{+1}, \phi_1$ are predictor coefficients. Intuitively, the first three terms exploit smoothness in the lowpass band, while the last term models correlation between adjacent highpass coefficients. These coefficients were originally obtained by fitting a local linear model to natural and medical images so as to minimize the mean-square prediction error of $d_1[n]$ given neighboring lowpass differences and the next highpass sample. The fitted values for $(\beta_{-1}, \beta_0, \beta_{+1}, \phi_1)$ are fixed and do not depend on the image, allowing the transform to remain perfectly reversible prior to quantization. For our implementation we use the coefficient set

$$\beta_{-1} = -1, \quad \beta_0 = 2, \quad \beta_{+1} = -1, \quad \phi_1 = 1,$$

which is the recommended configuration from [17].

To preserve integer reversibility, we do not subtract $\hat{d}_1[n]$ directly. Instead, we perform a quantized correction using an integer denominator $K = 2^k$:

$$\text{pred}[n] = \left\lfloor \frac{\hat{d}_1[n]}{K} \right\rfloor, \quad (23)$$

and define the residual

$$d[n] = d_1[n] - \text{pred}[n]. \quad (24)$$

The final transformed highpass component is thus $d[n]$, and $d_1[n]$ never needs to be stored explicitly beyond the prediction step.

5) *Inverse S+P and exact reconstruction*: The inverse transform proceeds in the reverse order of the forward transform. During the inverse of a one-dimensional pass, we assume that $s[n]$ and $d[n]$ are available. We reconstruct $d_1[n]$ by reapplying the same predictor in reverse:

- 1) Visit indices in descending order $n = N/2 - 1, \dots, 0$, ensuring that $d_1[n+1]$ is already reconstructed when needed.
- 2) For each n , compute the same predictor numerator using current $s[\cdot]$ and $d_1[n+1]$.
- 3) Recompute $\text{pred}[n] = \left\lfloor \hat{d}_1[n]/K \right\rfloor$ and add it back:

$$d_1[n] = d[n] + \text{pred}[n]. \quad (25)$$

Once $d_1[n]$ is recovered for all n , the inverse S transform recovers the original samples $c[2n]$ and $c[2n+1]$ exactly. The crucial point is that the predictor and its quantization are perfectly reproducible during inversion, so no information is lost.

6) *Frequency-Domain Interpretation*: Although the S+P transform is defined via local integer operations, its behavior is illuminating when viewed in the frequency domain. If we ignore truncation for the sake of analysis and treat the operations as linear, the mapping from the original sequence $c[n]$ to the highpass residual $d[n]$ can be represented as a linear time-invariant filter with a frequency response $F(e^{j\omega})$.

The S stage alone already suppresses low frequencies in the highpass band, but it does so relatively mildly; the resulting highpass spectrum still exhibits considerable low-frequency energy due to the piecewise-constant nature of the Haar-like filter. By embedding prediction in the transform, S+P effectively implements an additional highpass filter: the predictor attempts to explain away low-frequency variations in $d_1[n]$ using neighboring lowpass differences and adjacent highpass values. When the predictor coefficients are chosen appropriately, $|F(e^{j\omega})|$ exhibits significantly stronger attenuation at low frequencies than the basic S transform.

There is, of course, a trade-off. Stronger low-frequency attenuation generally comes at the cost of some amplification in higher frequencies. For smooth, low-noise images, this trade-off is supposedly advantageous: the additional whitening reduces the variance and first-order entropy of $d[n]$, improving compressibility. For highly textured or noisy images, it may be beneficial to temper the predictor so as not to over-amplify

high-frequency content. Said and Pearlman demonstrate that, in practice, a small set of carefully tuned predictors can work well across a broad range of natural and medical images [17].

7) *Complexity and Runtime Considerations*: From an algorithmic standpoint, the S+P transform has the same asymptotic complexity as the underlying S transform. For an image of size $W \times H$, a single S pass along rows and columns is $O(WH)$: every pixel participates in a constant number of operations. The P stage introduces only a small constant-factor overhead: each highpass coefficient requires a few extra integer operations for the predictor numerator and a single integer shift.

More explicitly:

- S stage: for each pair $(c[2n], c[2n+1])$, we compute one difference and one sum with a shift, $O(1)$ per pixel.
- P stage: for each highpass index n , we access a small fixed stencil of lowpass samples and one neighboring highpass value, perform a handful of integer multiplies/adds, and one right shift by a fixed number of bits, again $O(1)$ per highpass coefficient.

Because the transform is applied recursively only to the LL subband, which shrinks by a factor of four at each level, the total work over all levels of the pyramid remains $O(WH)$. Compared to block DCT or longer-tap wavelets, S+P remains light-weight, fully in-place, and integer-only; in our pipeline measurements, entropy coding and I/O dominate total runtime.

F. Quantization

1) *Quantization Theory*: Quantization is the lossy process of reducing the bits needed to store data by dividing from a larger range of values to a smaller range of values. By decreasing the range used to represent the data, entropy is lowered, as multiple values pre-quantization may be mapped to one value during quantization. With this mapping being many-to-one, this operation cannot be reversed, showing that quantization is necessarily lossy. With the goal of transforms being to decrease entropy while ensuring perfect reconstruction of an image, quantization is used to further improve compression ratio beyond entropy-bound compression used in lossless algorithms, sacrificing image fidelity in the process. Fig. 14 depicts the effect of increasing levels of quantization and the associated loss of image quality, along with compression ratio of the compressed file as defined in I.

A central component to quantization is step size Δ , which points to the ranges of pre-quantized values that would be quantized to one resulting value. With R representing the range of values and M the number of steps, step size is given by:

$$\Delta = \frac{R}{M} \quad (26)$$

To illustrate the role of step size, consider an 8-bit signal whose values lie in the range 0–255. A step size of $\Delta = 2$ would quantize that range into 128 steps:

$$2 = \frac{256}{M} \quad (27)$$

$$M = 128 = 2^7 \quad (28)$$

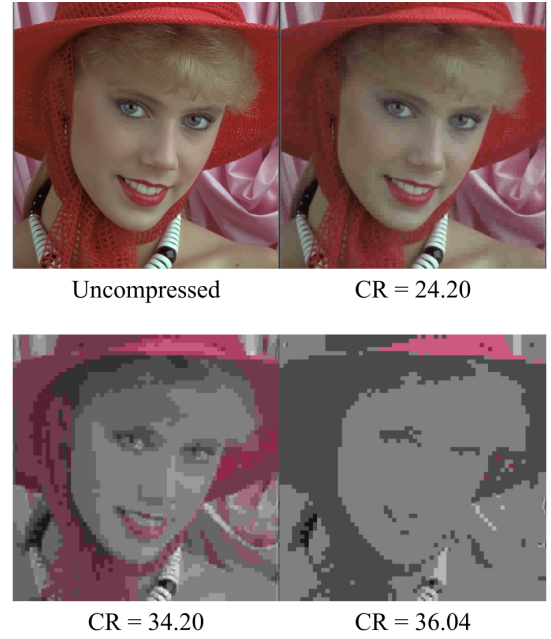


Fig. 14: Different levels of quantization on sample image with resulting compression ratio.

Thus, the output symbols of this example quantizer can be represented using 7 bits per value, before the following entropy encoding.

In our study, a relatively simple base quantization method was devised based on the JPEG Compression Standard. Quantization was applied on each chunk via division by a defined quantization matrix associated with each transform. DCT and DFT utilized a quantization matrix specified by the JPEG Compression Standard, whereas other transforms had quantization matrices calculated based on transformed data ranges.

Because the JPEG-specified quantization matrix is labeled as resulting in a 50% reduction in quality, the quantization methods for the Haar and S+P transforms aimed to reduce the theoretical bit-representation of the image from 8 bits to 7 bits. For the Haar transform, this was done by calculating the data ranges for each quadrant of each transformed chunk and choosing the appropriate step sizes based on $M = 128$ to achieve a basic quantization matrix. A unique quantization method was created for the S+P transform.

2) *Quantization for the S+P Transform*: For lossy compression, we introduce bandwise scalar quantization on top of the S+P coefficients. Because S+P already segregates information into subbands of differing perceptual importance (LL vs. HL/LH/HH), it is natural to assign different step sizes to each band rather than using a uniform matrix.

a) *Bandwise and level-dependent parameterization.*: Our implementation exposes a small set of quantization parameters in a `QuantParams` structure:

- $q_{LL}, q_{HL}, q_{LH}, q_{HH}$: base step sizes for the four subbands,
- `deadzone`: a dimensionless multiplier for the central dead-zone,

- *scale*: a global scale factor applied uniformly across bands,
- *level_gamma*: a geometric multiplier applied per pyramid level.

From these we compute per-level step sizes

$$q_{\text{band}}^{(L)} = \lfloor \text{scale} \cdot \text{level_factor}(L) \cdot q_{\text{band}} \rfloor, \quad (29)$$

$$\text{level_factor}(L) = \begin{cases} \text{level_gamma}^L, & \text{if } \text{level_gamma} > 0, \\ 1, & \text{otherwise.} \end{cases} \quad (30)$$

for each band and level L . In practice, the LL band at coarse levels is often left unquantized or lightly quantized, while higher-frequency bands (HL/LH/HH) receive progressively larger steps, reflecting their lower perceptual weight. Because all S+P coefficients are integers, these step sizes are also rounded to integers, which can further couple quantizer behavior to the underlying coefficient distribution. As in the Haar case, the choice of *deadzone* is made empirically and tuned based on observed rate-distortion behavior.

b) *Dead-zone scalar quantization.*: Within each sub-band, we apply a standard dead-zone scalar quantizer. Given a coefficient x and step size Δ , we compute

$$q = \begin{cases} 0, & |x| < \text{deadzone} \cdot \Delta, \\ \text{sign}(x) \left\lfloor \frac{|x|}{\Delta} \right\rfloor, & \text{otherwise,} \end{cases} \quad (31)$$

and reconstruct via $\hat{x} = q\Delta$ during decoding. The enlarged dead-zone encourages small-magnitude coefficients—especially in the highpass bands—to be quantized to zero, producing longer zero-runs that the entropy coder can compress efficiently.

Because the S+P transform tends to reduce the variance and entropy of its highpass residuals, relatively simple scalar quantizers can still achieve a wide range of operating points. Moreover, the explicit bandwise and level-dependent parameters allow us to adjust $(q_{LL}, q_{HL}, q_{LH}, q_{HH})$ and *level_gamma* to probe different rate-distortion regimes without modifying the transform itself.

G. Entropy Coding

Entropy coding is a class of compression algorithms to occur last in the pipeline. These algorithms are given their name because their effectiveness is bounded by the entropy of incoming data. In the JPEG Compression Standard, a combination of Run-Length Encoding (RLE) and Differential Pulse Code Modulation (DPCM) is used, whose results are then fed into a Huffman Coder. The areas where these algorithms are used can be observed in Fig. 15, with DPCM being used on the top left value of each chunk across all chunks in a channel (highlighted in green), and RLE being used on the remaining values in a single chunk, with the order in which values are encoded showed by the purple arrow.

Run-Length Encoding, as the name suggests, takes advantage of "runs" of data, storing the number of time a certain value or group of values repeats rather than storing every

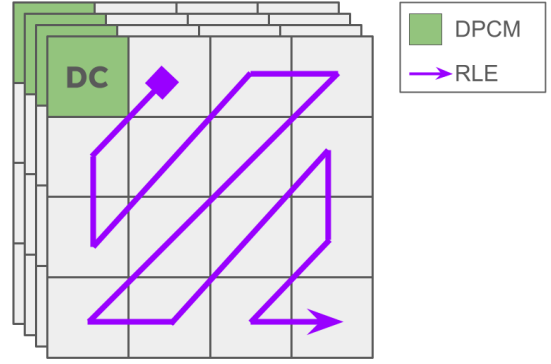


Fig. 15: Visual of different entropy encoding algorithms used for DCT as specified by JPEG Compression Standard.

single repetition. In the case of the JPEG standard, RLE is used to group runs of zeroes in the AC components of DCT together, which appear in the data as a result of heavy quantization of DCT's high-frequency coefficients. So, each piece of data is encoded as a pair of integers, with the first being the number of preceding zeroes (up to 15) and the second being the following non-zero number (or a zero if there are 15 preceding zeroes). As an example, the string of values

$$0, 0, 0, 4, 0, 12, 0, 0, 8, 2.$$

would be encoded to

$$(3, 4); (1, 12); (2, 8); (0, 2).$$

While RLE is used for AC components, DPCM is used for DC components. At its core, DPCM is purely a differential coding algorithm, meaning it stores only the differences between values rather than the actual values themselves. This algorithm is most effective when adjacent values are correlated, which is the case for DC coefficients of adjacent image chunks (see "why images section"). By storing solely the differences between values, DPCM effectively stores values from the derivative function of the unencoded data, provided the unencoded data is correlated by some function itself. Thus, the efficiency of this algorithm depends much on the hope that the entropy of the derivative function is lower than that of the original function. Of course, to make data reconstruction possible, the first original value is stored, similarly to how integral calculation requires the constant C . To reconstruct the data, the differences are added to the first original value in sequence. For some sequence $[x]$ with difference sequence $[d]$:

$$\begin{aligned} d_0 &= x_0 \\ d_1 &= x_1 - x_0 \\ d_n &= x_n - x_{n-1} \end{aligned} \quad (32)$$

and for reconstruction:

$$\begin{aligned} x_0 &= d_0 \\ x_1 &= x_0 + d_1 \\ x_n &= x_{n-1} + d_n \end{aligned} \quad (33)$$

Related to DPCM is predictive coding, which uses previous values to predict future values and stores the difference between the actual value and the predicted value, known as prediction error. In cases when the data is highly correlated, predictions can be very accurate, causing the prediction error to be even smaller than differences calculated using DPCM, lowering entropy even further. Because of this, DPCM was substituted out for a linear predictive coder in this project pipeline. Similarly to DPCM, given some sequence $[x]$, we have predicted sequence $[p]$, with difference sequence $[d]$, where the difference sequence is stored in final data:

$$\begin{aligned} p_n &= f(x_{n-1}, x_{n-2}, \dots, x_{n-k}) \\ d_n &= x_n - p_n \end{aligned} \quad (34)$$

where function $f()$ is a linear regression model using k previous values.

Luca

1) *Huffman Coding*: Huffman coding is an entropy encoding algorithm that serves a crucial role in the final stage of image compression pipelines. The purpose of Huffman coding is to assign variable-length binary codes to symbols based on their frequency of occurrence, with more frequent symbols receiving shorter codes and less frequent symbols receiving longer codes. This approach enables efficient representation of data by reducing the average number of bits required to encode each symbol.

In the context of image compression, Huffman coding is particularly useful because it can achieve near-optimal compression when the symbol frequencies are known. While the transform coding steps (such as DWT, DCT, and DFT) and quantization all aim at reducing the entropy of the image data by concentrating energy and removing redundancy, these steps alone do not actually compress the data if written out directly. Even after these transformations, the data still requires 8 bits per channel per pixel to represent each value. Huffman coding addresses this limitation by exploiting the non-uniform distribution of values in the transformed and quantized data, allowing frequently occurring values to be represented with fewer bits and less frequent values with more bits. This results in a reduction of the total number of bits needed to represent the data in a near-optimal way, at least when comparing to character representation with knowledge only of the character frequencies.

IV. RESULTS

A. Experimental Setup

To compare each transform, we chose 4 unique datasets all with different types of images. Sample images for all of the datasets can be seen in Fig. 16.

- 1) **Kodak** - Includes general pictures of people and objects. We included this to test how each transform performed on normal images.
- 2) **Aerial Images** - As the name suggests, contains images taken from the air of large sections of landscape. We expected many transforms like Haar and S+P to do

well on these images because of their consistent global structure,

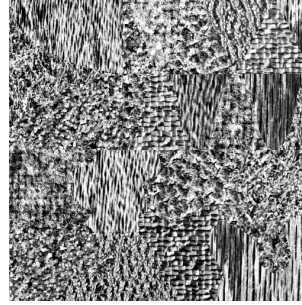
- 3) **Textures** - The inverse of Aerial Images, this includes close-up textures of a variety of surfaces and materials. This dataset is interesting because it includes some images with good global consistency, but also many images with much more sporadic textures that vary greatly even at a small scale.
- 4) **Icons** - This includes low-resolution images of a variety of various digital icons. We chose this to test since the sharp edges and transitions in color challenge the transforms, especially those like DCT and DFT.



(a) Sample **Kodak** image



(b) Sample **Aerial** image



(c) Sample **Texture** image



(d) Sample **Icon** image

Fig. 16: A sample image from each of the 4 datasets.

B. Metrics

To evaluate the behavior of each transform within our compression pipeline, we report several standard metrics drawn from image coding. These metrics quantify both reconstruction fidelity and the effectiveness of our bitstream reduction steps.

a) *Mean Squared Error (MSE)*.: Given an original image I and its reconstruction $\hat{I}$, the MSE is

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (I_i - \hat{I}_i)^2, \quad (35)$$

where N is the number of pixels. Lower MSE indicates a reconstruction closer to the original.

b) *Peak Signal-to-Noise Ratio (PSNR)*.: PSNR provides a logarithmic measure of fidelity, defined as

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{ dB}, \quad (36)$$

1. Parametric Performance Curves by Quantization Scale (Transform: All)

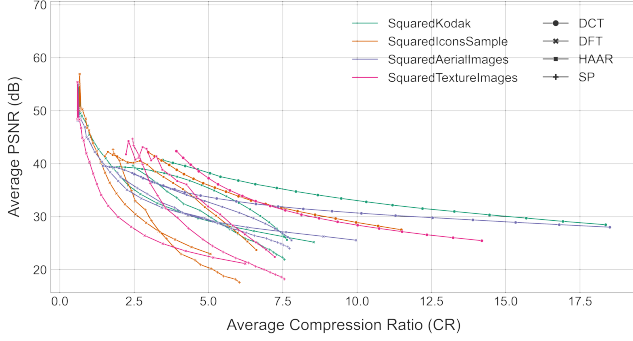


Fig. 17: Compressed image similarity based on compression ratio by transform and dataset.

assuming 8-bit images. Higher PSNR corresponds to higher reconstruction quality. We use PSNR as the primary distortion metric in all rate–distortion plots.

c) *Compression Ratio (CR)*.: As a reminder, we define the compression ratio as

$$CR = \frac{\text{uncompressed size}}{\text{compressed size}}.$$

Unless otherwise noted, all CR values in the rate–distortion plots refer to the final bitstream after entropy coding. For completeness, we also report the pre-entropy (“direct”) CR in one diagnostic figure to examine how much gain each transform receives from the entropy coder.

We will also continue to use Shannon’s definition of entropy discussed in Section II. It gives an accurate picture of how well each transform successfully decorrelated the input image.

Together, these metrics provide the basis for the rate–distortion evaluations reported in Section IV.

C. Collective Results

Firstly, as shown in fig. 17, all transforms’ had their quality of compressed images compared against compression ratio. Quality of compressed images, measured by the similarity between compressed and original image, or PSNR (db), is much higher for DCT, colored in blue, than all other transforms at all compression levels, with DCT additionally reaching higher compression levels than any other transform. Fig. 18 explores compression ratio by transform, with each box per transform representing compression ratio data with or without entropy coding used. For DCT and DFT, entropy coding generally increases compression ratio, with significant increases to the highest compression ratios achieved. On the other hand, Haar and SP see a flat decrease in compression ratio when entropy coding is used.

D. DWT Results

The results demonstrate that the Discrete Wavelet Transform (DWT) using the Haar wavelet basis exhibits varying performance across different image datasets. Notably, the method performed worse on the Texture Images and Icons Images

Compression Ratio of output with and without Entropy Coding by Transform

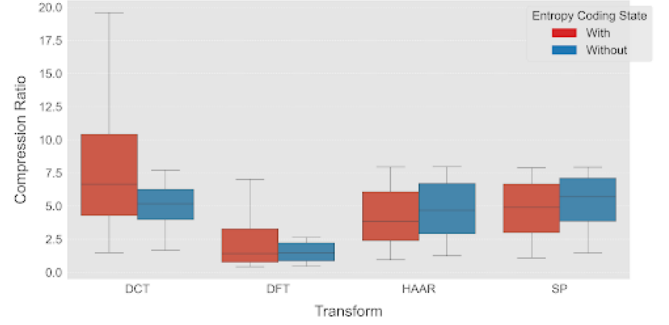


Fig. 18: Effect of entropy encoding on compression ratio for all transforms

1. Parametric Performance Curves by Quantization Scale (Transform: HAAR)

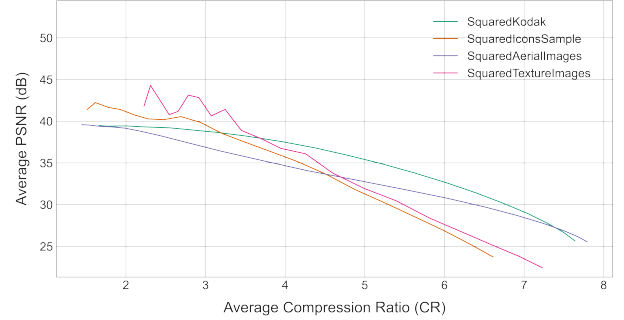


Fig. 19: Performance of the Haar wavelet transform across different datasets

datasets as quantization became more aggressive compared to other image types as can be see in Figure 19.

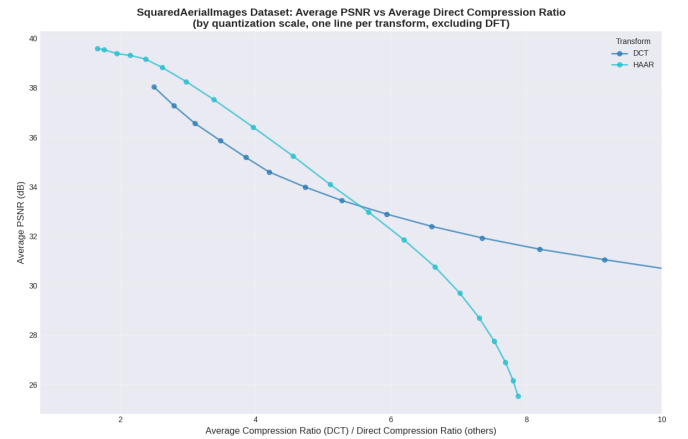


Fig. 20: Performance comparison of Haar and DCT on aerial images.

Figure 20 shows that the Haar wavelet transform performs slightly better than DCT on aerial images at smaller compression ratios.

In contrast, Figure 21 demonstrates that Haar performs worse than DCT on Kodak images across all compression ratio scenarios.

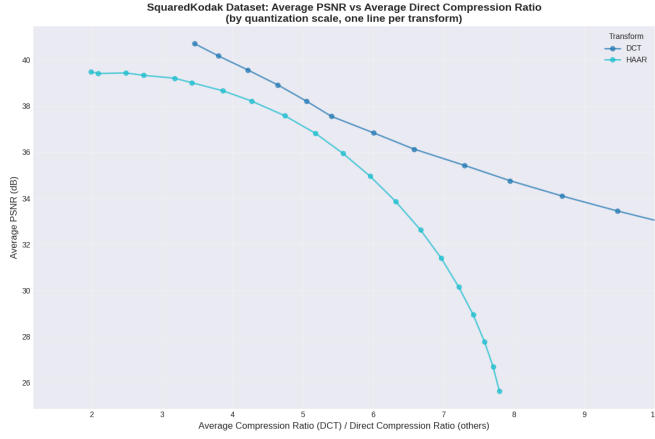


Fig. 21: Performance comparison of Haar and DCT on Kodak images.

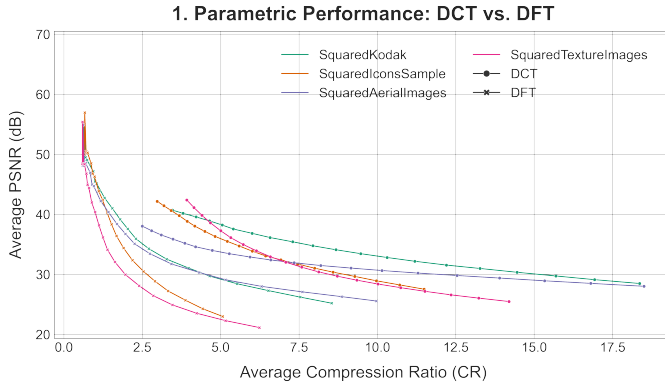


Fig. 22: Different levels of quantization on sample image with compression ratio.

E. DCT Results

We claimed in III-D4 that DCT is superior to DFT for image compression. To see this clearly, Fig. 22 shows how DCT achieves much higher compression ratios at equivalent PSNR values across all datasets. For example, at a PSNR of 30 dB, the DFT achieves $CR \approx 5$ across all datasets, while DCT has $CR \approx 10$, even closer to 15 for Aerial Images.

Notably, in IV-C it appears that the DCT outperforms all transform algorithms, not just the DFT. This comparison is slightly unfair since an additional step RLE was added to the entropy coding for DCT as described in III-G. Fig. 23 shows how the compression ratios change without RLE present for a more fair comparison.

This provides an excellent example of how compression algorithms can work together to achieve maximum compression for input data. Since the DCT is part of the well established JPEG pipeline, there are lots of extra tools to push the compression to the limit. Likely given more time, other techniques could be implemented to further improve algorithms like DWT and S+P as well.

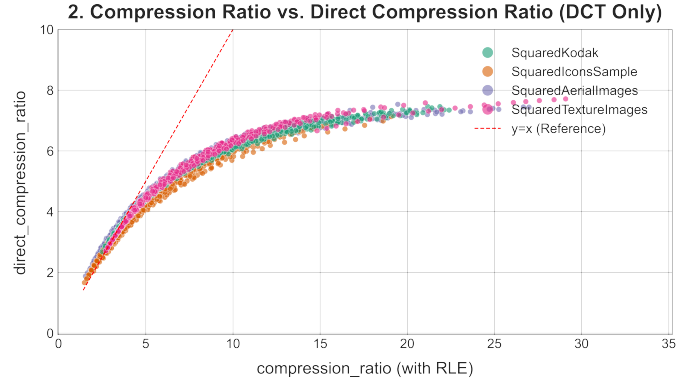


Fig. 23: Different levels of quantization on sample image with compression ratio.

1. Parametric Performance Curves by Quantization Scale (Transform: SP)

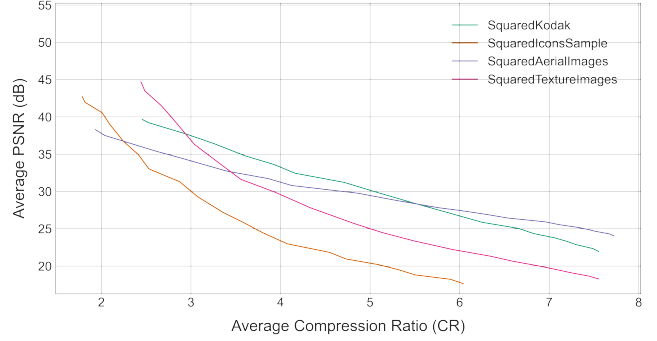


Fig. 24: S+P parametric performance: average PSNR vs. average compression ratio.

F. S+P Results

Fig. 24 shows, for each squared dataset, the average PSNR (in dB) as a function of the average compression ratio (CR) obtained by sweeping the S+P quantization scale. As expected, all four curves exhibit a monotone tradeoff: increasing CR is accompanied by decreasing PSNR. The datasets, however, occupy different regions of the CR axis. Kodak, AerialImages, and TextureImages all span roughly $CR \approx 2-8$, whereas IconsSample is confined to about $CR \approx 1.7-6$, reflecting that these very small, simple images cannot be pushed to the same nominal compression ratios. At the low-CR end (around $CR \approx 2$), TextureImages achieves the highest average PSNR (around 45 dB), with IconsSample close behind, and Kodak and AerialImages clustered around 38–39 dB. As CR increases, all curves slope downward, but with noticeably different steepness. The Kodak and AerialImages curves remain relatively close together, maintaining PSNR in the low 30 dB range at $CR \approx 4$ and still above 25 dB at $CR \approx 7$. TextureImages falls somewhat faster but stays competitive through most of the range. IconsSample begins near the top-left of the plot with high PSNR at low CR, but its curve is the steepest: by $CR \approx 5$ it has dropped to around 20 dB, and it does not extend into the very high-CR regime where the other datasets continue.

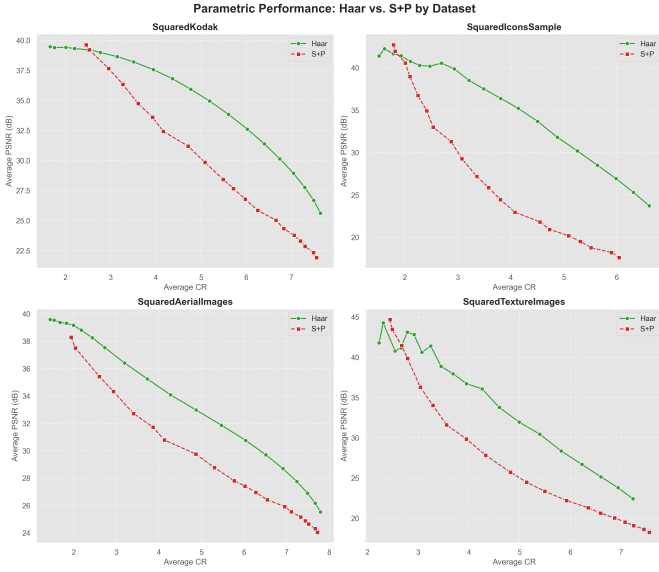


Fig. 25: S+P parametric performance: average PSNR vs. average compression ratio.

Overall, these curves show that S+P exhibits the same dataset ordering we observed in the Collective Results, with natural-image datasets clustering together and IconsSample behaving as a low-CR, high-PSNR regime.

Fig. 25 directly compares Haar and S+P on the same CR–PSNR axes for each squared dataset. In all four panels, both transforms exhibit the expected monotone tradeoff between compression ratio and average PSNR, but the two curves separate as CR increases. For Kodak and AerialImages, the Haar and S+P curves start close together at low compression ratios (around $CR \approx 2$ with PSNR near 39–40 dB), after which the S+P PSNR decreases more rapidly, yielding several dB lower PSNR than Haar at medium and high CR. A similar pattern appears for IconsSample and TextureImages: at the lowest CR values S+P is comparable to Haar (and slightly higher at one operating point for textures), but beyond roughly $CR \approx 2$ the S+P curve bends downward more steeply, while the Haar curve remains higher and extends further into the high-CR regime.

V. DISCUSSION

DCT’s superior performance in overall compression is due to the fact that the JPEG Compression Standard has entropy coding algorithms tailor-made to be effective for quantized DCT. Because of the combination of RLE and heavy quantization of high-frequency components, which results in an abundance of zeroes, DCT was able to reach much higher compression ratios than other transforms. Of course, without the clever quantization of components less perceptible to humans, the use of RLE would not be able to reach its full potential. The frequent long runs of zeroes allow many values to be represented by just two. With the Haar and S+P transforms, a flat predictive encoder was used, which does not decrease the amount of values needed to be represented. Thus,

there is a minimum of one bit per value, whereas in DCT, RLE is able to theoretically break past this minimum because a long run of zeroes is represented with just one value.

DFT, although not as space-efficient when measured against preserving image quality, saw an increase in compression ratio similar to that of DCT when using JPEG-specified entropy coding techniques, which speaks to its structural similarity to DCT. The decrease in compression ratio when using a flat linear predictive coder for the Haar and S+P transforms further shows the importance of the structure of the transform. DPCM and predictive coders heavily rely on correlation between values, which points to the fact that the order in which the values were encoded with predictive coding after the Haar and S+P transforms were not correlated, causing the two transforms to perform worse in compression ratio when entropy coding was used. So, without taking into account the order in which values may be correlated, which is heavily dependent on transform structure, algorithms such as DPCM and predictive coding may actually decrease compression performance.

By taking advantage of the structure of DCT in the quantization and entropy coding steps, the JPEG Compression Standard performs much better than the other transforms explored in this project. In the following subsections, we discuss the implications of results for each individual transform.

A. Discussion: DWT

As shown in Figure 19 in the Results section, the Discrete Wavelet Transform (DWT) using the Haar wavelet basis exhibits degraded performance on the Texture Images and Icons Images datasets as quantization becomes more aggressive. This degradation in performance can be attributed to the finer details present in these datasets, which are more susceptible to being quantized away during the compression process. The quantization step in the compression pipeline tends to discard high-frequency components that represent fine textures and intricate icon details, leading to a more significant loss of visual information in these particular datasets compared to other image types.

The comparison between Haar and DCT on different image types, as shown in Figures 20 and 21, reveals important characteristics of the Haar wavelet transform. Haar performs better on aerial images because it operates on a global image scale, which is particularly effective for aerial images that typically contain large patches of very similar color, such as oceans or deserts. On the other hand, Haar performs worse on Kodak images since it is less optimized than DCT in the JPEG pipeline, as discussed previously. Additionally, Kodak images generally do not contain large global features of the same color as found in aerial images, which limits the effectiveness of Haar’s global approach.

B. Discussion: S+P

Figs. 25 show that, in our current pipeline, the S+P transform does not surpass the Haar wavelet in rate–distortion performance. S+P remains competitive with Haar at very low bit-rates, but its PSNR degrades more quickly as quantization

becomes coarser across all datasets. This behavior aligns with several structural differences between the two transforms and with the way our quantizer interacts with their coefficient statistics.

First, S+P modifies the basic Haar lifting step by introducing an in-loop prediction on the highpass branch. After computing the integer lowpass and preliminary detail coefficients (s, d_1) , S+P replaces $d_1[l]$ with a residual

$$d[l] = d_1[l] - \left\lfloor \frac{\text{pred}(s, d_1[l+1])}{2^{\text{shift}}} \right\rfloor,$$

where pred is a fixed linear combination of neighboring lowpass differences and the next detail value. This predictor effectively acts as an additional highpass filter: it substantially reduces low-frequency leakage in the detail band whenever the correlation condition $\rho > 1/3$ holds, thereby lowering the variance and first-order entropy of $d[n]$. While this whitening is advantageous at the transform stage, it also concentrates more energy into the highest spatial frequencies, which are precisely the coefficients most heavily attenuated by our coarse scalar quantizer. Moderate step sizes that Haar tolerates well therefore produce noticeably larger PSNR losses under S+P.

Second, unlike Haar, which produces subbands with simple and predictable variance structure, the S+P transform applies additional prediction on the highpass branch. This prediction further suppresses low-frequency leakage in the detail bands, effectively shifting more residual energy toward the highest frequencies. Although this reduces the first-order entropy of the detail coefficients—an advantage for lossless or near-lossless coding—it also increases the sensitivity of these coefficients to coarse scalar quantization, which disproportionately attenuates high-frequency components. Because our quantization parameters were tuned heuristically and not optimized for S+P’s predictor-adjusted subbands, the resulting step sizes do not match the actual coefficient distributions well, contributing to the larger loss in PSNR at matched bit-rates.

Finally, Fig. 18 shows that although S+P achieves some of the highest direct compression ratios before entropy coding, the additional gain from our backend is smaller for S+P than for Haar. Our entropy coder consists of a simple JPEG-style differential predictor followed by symbolwise Huffman coding; it does not perform run-length coding or adaptive context modeling. Because S+P already removes much of the spatial correlation and flattens the coefficient structure, it leaves less residual redundancy for this backend to exploit. Haar, by comparison, retains more correlation patterns that Huffman coding can capitalize on, leading to larger post-entropy-coding improvements.

Taken together, these factors explain why S+P appears promising at the transform stage, with lower entropy and strong direct compression, yet yields a weaker end-to-end CR–PSNR tradeoff than Haar under our current JPEG-style pipeline. With predictor-aware quantization and a more sophisticated entropy model, S+P’s relative position could plausibly change.

C. Discussion: DCT + DFT

We have already established that the DCT outperforms the DFT across all datasets. However, it is worth noting that due to the similar nature of the transforms, their performance is relatively consistent across datasets. In Fig. 22, we can see that they both perform better on the Kodak and Aerial Images datasets, and worse on Textures and Icons. This is consistent with their theory since the images in Textures and Icons are both more likely to have more rapid changes in pixel values corresponding to higher frequency waveforms.

We briefly mentioned in results how some parts of entropy coding like RLE were specifically designed for the DCT. Fig. 23 shows the gap between the expected compression ratio based purely on the entropy of the transformed image versus the actual compression ratio achieved. At lower quantization scales, the RLE makes little difference as there are still many nonzero values. However, as quantization is made more aggressive the returns from RLE increase massively as there are longer and longer runs of 0s.

APPENDIX A THEORETICAL FOUNDATION FOR HAAR WAVELET TRANSFORM

a) Vector Spaces and Nested Approximations: The foundation of wavelet transform can be understood through the definition of a set of nested vector spaces. We can represent any signal as piecewise-constant functions defined on the half-open interval $[0, 1)$. We utilize vector spaces, denoted as V^j , to represent these functions at different resolutions.

The space V^0 contains functions that are constant over the entire interval $[0, 1)$. A one-pixel image is an element of this space. The space V^1 contains functions with two constant pieces over the intervals $[0, 1/2)$ and $[1/2, 1)$. The space V^2 contains functions with four constant pieces over the intervals $[0, 1/4)$, $[1/4, 1/2)$, $[1/2, 3/4)$, and $[3/4, 1)$. Generalizing this, the space V^j includes all piecewise-constant functions defined on $[0, 1)$ with constant pieces over each of 2^j equal subintervals:

$$\left[0, \frac{1}{2^j}\right), \left[\frac{1}{2^j}, \frac{2}{2^j}\right), \dots, \left[\frac{2^j - 1}{2^j}, 1\right)$$

A crucial property of these spaces is their nested structure. Because a function that is constant over a large interval is inherently constant over any subdivision of that interval, every vector in V^j is also contained in V^{j+1} . For example, a function with two intervals can be described as a function with four intervals where pairs of intervals have identical values. This creates a nested set of spaces:

$$V^0 \subset V^1 \subset V^2 \subset \dots$$

This nested structure is essential for the transform and motivates our representation of signals as this family of vector spaces.

b) *Scaling Functions (The Basis for V^j):* The basis functions for the spaces V^j are known as **scaling functions**, usually denoted by ϕ . A natural and simple basis for V^j is derived from the scaled and translated "box" function $\phi(x)$, defined as:

$$\phi(x) := \begin{cases} 1 & \text{for } 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases}$$

For a specific resolution level j , the basis functions ϕ_i^j are defined by scaling and translating $\phi(x)$:

$$\phi_i^j(x) := \phi(2^j x - i), \quad i = 0, \dots, 2^j - 1$$

A visualization for this basis in V^3 is shown in Figure 5(a).

c) *Inner Products and Orthogonality:* To define wavelets, we must first define an **inner product** to measure the relationship between functions. The standard inner product for two elements $f, g \in V^j$ is defined as:

$$\langle f | g \rangle := \int_0^1 f(x)g(x)dx$$

Using this inner product, we define a new vector space W^j as the **orthogonal complement** of V^j in V^{j+1} . In other words, we will let W^j be the space of all functions in V^{j+1} that are orthogonal to all functions in V^j under the chosen inner product. Informally, we can think of the wavelets in W^j as a means for representing the parts of a function in V^{j+1} that cannot be represented in V^j .

A collection of linearly independent functions $\psi_i^j(x)$ spanning W^j are called **wavelets**. These basis functions have two important properties:

- 1) The basis functions ψ_i^j of W^j , together with the basis functions ϕ_i^j of V^j , form a basis for V^{j+1} .
- 2) Every basis function ψ_i^j of W^j is orthogonal to every basis function ϕ_i^j of V^j under the chosen inner product.

Thus, the "detail coefficients" are really coefficients of the wavelet basis functions.

d) *The Haar Wavelet Basis Functions:* The wavelets corresponding to the box basis are called **Haar wavelets**. The "mother wavelet" $\psi(x)$ is defined as:

$$\psi(x) := \begin{cases} 1 & \text{for } 0 \leq x < 1/2 \\ -1 & \text{for } 1/2 \leq x < 1 \\ 0 & \text{otherwise} \end{cases}$$

Similar to the scaling functions, the Haar wavelets for a specific level j are generated by scaling and translating this function:

$$\psi_i^j(x) := \psi(2^j x - i), \quad i = 0, \dots, 2^j - 1$$

Figure 5(b) illustrates the relationship between the scaling functions (blue) and the Haar wavelet basis functions (red) for W^2 . The wavelets capture the detail information that cannot be represented in the lower-resolution space V^2 .

e) *Generalizing to Other Basis:* This illustration of this specific vector space of piecewise constant functions is unique to the Haar Wavelets, however we can similarly define other vector spaces and inner products to work with different wavelets. The essential part illustrated by Haar is that the vector spaces must be nested and the definition of the wavelet space as the orthogonal complement of the scaling functions inside the next resolution level. This general idea of energy compactation into the lower resolution's basis and using the wavelets as the detail coefficient remain the same regardless of the basis. Figure 26 demonstrates alternative wavelet bases using linear and quadratic splines.

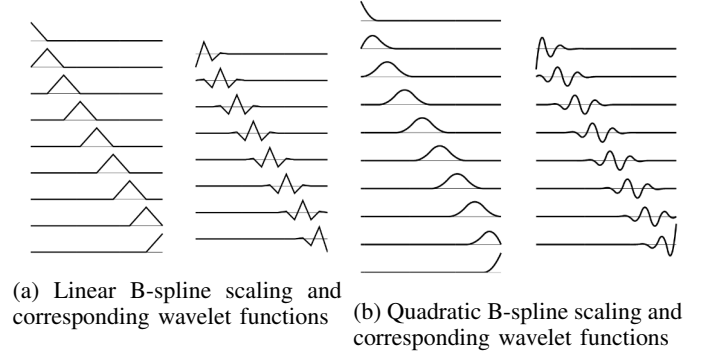


Fig. 26: Alternative wavelet bases demonstrating different spline functions: linear and quadratic splines, showing alternatives to the Haar wavelets. Figures adapted from [23]

It is somewhat standard to treat the space as some space of functions from the half open interval $[0, 1)$ when dealing with images because we can use the standard inner product previously defined very effectively but there is nothing that strictly requires such representation.

APPENDIX B

THE "CORRELATION $> 1/3$ " CONDITION IN HAAR AND S+P

Many analyses of the Haar and S+P transforms rely on the assumption that adjacent samples in natural images exhibit a positive correlation greater than $1/3$. This condition arises from a simple variance calculation of the Haar lifting step.

Let $c[n]$ be a stationary one-dimensional signal with variance σ^2 and correlation coefficient

$$\rho = \frac{\text{Covar}(c[n], c[n+1])}{\sigma^2}.$$

In the 1D Haar transform, each pair $(c[2n], c[2n+1])$ is mapped to an average and a difference:

$$s[n] = \frac{c[2n] + c[2n+1]}{2}, \quad d_1[n] = c[2n+1] - c[2n].$$

Assuming stationarity, the variances of these components are:

$$\text{Var}(s) = \frac{\sigma^2}{2} (1 + \rho), \quad \text{Var}(d_1) = 2\sigma^2 (1 - \rho).$$

For the transform to be variance-reducing (or at least not variance-increasing), we require that the total output variance not exceed the original variance of the pair:

$$\text{Var}(s) + \text{Var}(d_1) \leq 2\sigma^2.$$

Substituting the expressions above yields

$$\frac{\sigma^2}{2}(1 + \rho) + 2\sigma^2(1 - \rho) \leq 2\sigma^2,$$

which simplifies to

$$\rho > \frac{1}{3}.$$

Thus, when the correlation between neighboring samples exceeds $1/3$, the Haar averaging–differencing step reduces redundancy by lowering total variance. Since natural images typically exhibit correlations in the range 0.8–0.95, this condition is amply satisfied in practice, explaining the effectiveness of Haar-based and S+P transforms for decorrelation and compression.

APPENDIX C

S+P FREQUENCY DOMAIN INTERPRETATION

Although the S+P transform is defined via local integer operations, its behavior is illuminating when viewed in the frequency domain. If we ignore truncation for the sake of analysis and treat the operations as linear, the mapping from the original sequence $c[n]$ to the highpass residual $d[n]$ can be represented as a linear time-invariant filter with a frequency response $F(e^{j\omega})$.

The S stage alone already suppresses low frequencies in the highpass band, but it does so relatively mildly; the resulting highpass spectrum still exhibits considerable low-frequency energy due to the piecewise-constant nature of the Haar-like filter. By embedding prediction in the transform, S+P effectively implements an additional highpass filter: the predictor attempts to explain away low-frequency variations in $d_1[n]$ using neighboring lowpass differences and adjacent highpass values. When the predictor coefficients are chosen appropriately, $|F(e^{j\omega})|$ exhibits significantly stronger attenuation at low frequencies than the basic S transform.

There is, of course, a trade-off. Stronger low-frequency attenuation generally comes at the cost of some amplification in higher frequencies. For smooth, low-noise images, this trade-off is supposedly advantageous: the additional whitening reduces the variance and first-order entropy of $d[n]$, improving compressibility. For highly textured or noisy images, it may be beneficial to temper the predictor so as not to over-amplify high-frequency content. Said and Pearlman demonstrate that, in practice, a small set of carefully tuned predictors can work well across a broad range of natural and medical images [9].

APPENDIX D

THE FAST FOURIER TRANSFORM

As discussed in Section III-D4, a naive DFT implementation has a runtime of $\mathcal{O}(N^2)$ for an input signal $x = \{x_0, \dots, x_{N-1}\}$. The Fast Fourier Transform (FFT) significantly speeds up this process. The name “FFT” actually refers

to a whole family of algorithms that can solve a wider range of problems, but all of them take advantage of the symmetry in the complex plane to use a divide and conquer scheme. The FFT was first formally proposed by Cooley and Tukey in their 1965 paper [3], and the mathematical discussion here is inspired by Broughton [4]. We start with Equation 10 for the DFT Transform, but instead write it as

$$X_{(N/2)k_1+k_0} = \sum_{m=0}^{N-1} x_m (-1)^{k_1 m} e^{-2\pi i k_0 m / N}, \quad (37)$$

where $k_1 \in \{0, 1\}$ and $0 \leq k_2 < N/2$. Note that now when $k_1 = 0$ we are referring to the first $N/2$ coefficients, while when $k_1 = 1$ we are referring to the second half. If we then split the sum into its even and odd terms and simplify, we get that

$$X_{k_0} = F_1(k_0) + e^{-2\pi i k_0 / N} F_2(k_0) \quad (38)$$

and

$$X_{N/2+k_0} = F_1(k_0) - e^{-2\pi i k_0 / N} F_2(k_0), \quad (39)$$

where

$$F_1(k_0) = \sum_{m=0}^{N-1} x_{2m} e^{-2\pi i k_0 m / (N/2)} \quad (40)$$

and

$$F_2(k_0) = \sum_{m=0}^{N-1} x_{2m+1} e^{-2\pi i k_0 m / (N/2)}. \quad (41)$$

Note that $F_1(k_0)$ and $F_2(k_0)$ describe the DFT on the even and odd terms of the original signal x , respectively. In other words, to get the full length N DFT, one needs only to calculate the length $N/2$ DFT on the even and odd terms, then sum them to get the first $N/2$ terms for the full DFT (Eq. 38) and take their difference for the second $N/2$ terms (Eq. 39). This process can then be repeated recursively, further breaking down each half transform. The first step of this process is shown in Fig. 27. In this sense, the FFT is a classic divide and conquer algorithm, and an application of the Master Theorem can rigorously prove the $\mathcal{O}(N \log N)$ runtime that results. We refer a reader interested in the derivations of the relations in Equation 38 and Equation 39 once again to Broughton [4]. For an in-depth walkthrough of how the FFT can be implemented efficiently, see the CP algorithms page [2].

ACKNOWLEDGMENT

We would like to thank our advisor Layla Oesper for all her help throughout this project.

REFERENCES

- [1] C. A. Bouman, “JPEG Image Coding Laboratory Manual,” *Purdue University* Available: <https://engineering.purdue.edu/bouman/grad-labs/JPEG-Image-Coding/pdf/lab.pdf>.
- [2] CP-Algorithms contributors, “Fast Fourier Transform (FFT),” *CP-Algorithms*. Available: <https://cp-algorithms.com/algebra/fft.html>.
- [3] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [4] S. A. Broughton and K. Bryan, *Discrete Fourier Analysis and Wavelets: Applications to Signal and Image Processing*. Hoboken, NJ: John Wiley & Sons, 2009.

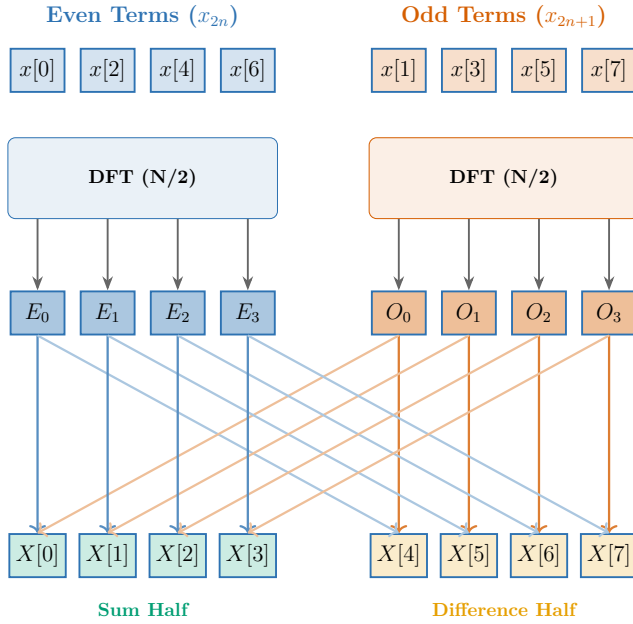


Fig. 27: The first recursive step in the FFT.

- [5] T. Chlubna and P. Zemčík, "Comparative survey of image compression methods across different pixel formats and bit depths," *Signal, Image and Video Processing*, vol. 19, no. 12, p. 981, Sep. 2025.
- [6] R. A. DeVore, B. Jawerth, and B. J. Lucier, "Image compression through wavelet transform coding," *IEEE Transactions on Information Theory*, vol. 38, no. 2, pp. 719–746, Mar. 1992.
- [7] J. Han *et al.*, "A Technical Overview of AV1," *Proceedings of the IEEE*, vol. 109, no. 9, pp. 1435–1462, Sept. 2021.
- [8] M. K. Ibraheem, "A Comprehensive Literature Review on Image and Video Compression: Trends, Algorithms, and Techniques," *IJETA*. [Online]. Available: <https://ijeta.org/journals/isi/paper/10.18280/isi.290307>.
- [9] U. Jayasankar, V. Thirumal, and D. Ponnuram, "A survey on data compression techniques: From the perspective of data quality, coding schemes, data type and applications," *Journal of King Saud University - Computer and Information Sciences*, vol. 33, no. 2, pp. 119–140, Feb. 2021.
- [10] S. A. Khayam, "The discrete cosine transform (DCT): theory and application," *Michigan State University*, vol. 114, no. 1, p. 31, 2003.
- [11] J. Lee, S. Cho, and M. Kim, "An End-to-End Joint Learning Scheme of Image Compression and Quality Enhancement with Improved Entropy Minimization," *arXiv preprint arXiv:1912.12817*, Mar. 2020.
- [12] A. Mali, A. Ororbia, D. Kifer, and L. Giles, "Neural JPEG: End-to-End Image Compression Leveraging a Standard JPEG Encoder-Decoder," *arXiv preprint arXiv:2201.11795*, Jan. 2022.
- [13] S. Mallat, *A Wavelet Tour of Signal Processing: The Sparse Way*, 3rd ed. Academic Press, 2009.
- [14] R. M. Rad, A. Attar, and A. Shahbahrami, "A predictive algorithm for multimedia data compression," *Multimedia Systems*, vol. 19, no. 2, pp. 103–115, Mar. 2013.
- [15] A. M. Raid, W. M. Khedr, M. A. El-dosuky, and W. Ahmed, "Jpeg Image Compression Using Discrete Cosine Transform - A Survey," *arXiv preprint arXiv:1405.6147*, May 2014.
- [16] S. R. D. Sivakumar, "Development of a Modified Predictive Coding Algorithm for High-Resolution Image Compression in Real-Time Structural Health Monitoring of Metallic Component," *Metallurgical and Materials Engineering*, vol. 31, no. 4, pp. 76–83, Apr. 2025.
- [17] A. Said and W. A. Pearlman, "Reversible image compression via multiresolution representation and predictive coding," *Visual Communications and Image Processing '93*, pp. 664–674, 1993.
- [18] A. Said and W. A. Pearlman, "Reversible image compression via multiresolution representation and predictive coding," in *Visual Communications and Image Processing '93*, vol. 2094. SPIE, Oct. 1993, pp. 664–674.

- [19] K. Sayood, *Introduction to Data Compression*, 4th ed. Elsevier, 2012.
- [20] C. E. Shannon, "A Mathematical Theory of Communication," *The Bell System Technical Journal*, vol. 27, pp. 379–423, 1948.
- [21] A. Skodras, *Discrete Wavelet Transform: An Introduction*. Jan. 2003.
- [22] D. H. Salesin, T. D. DeRose and E. J. Stollnitz, *Wavelets for Computer Graphics: A Primer, Part 1 in IEEE Computer Graphics and Applications*, vol. 15, no. 03, pp. 76–84, May 1995.
- [23] D. H. Salesin, T. D. DeRose and E. J. Stollnitz, *Wavelets for Computer Graphics: A Primer, Part 2 in IEEE Computer Graphics and Applications*, vol. 15, no. 04, pp. 75–85, July 1995.
- [24] G. K. Wallace, "The JPEG Still Picture Compression Standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, Feb. 1992.
- [25] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Transactions on Image Processing*, vol. 9, no. 8, pp. 1309–1324, Aug. 2000.